send_lark2 飞书消息服务-管理员手册

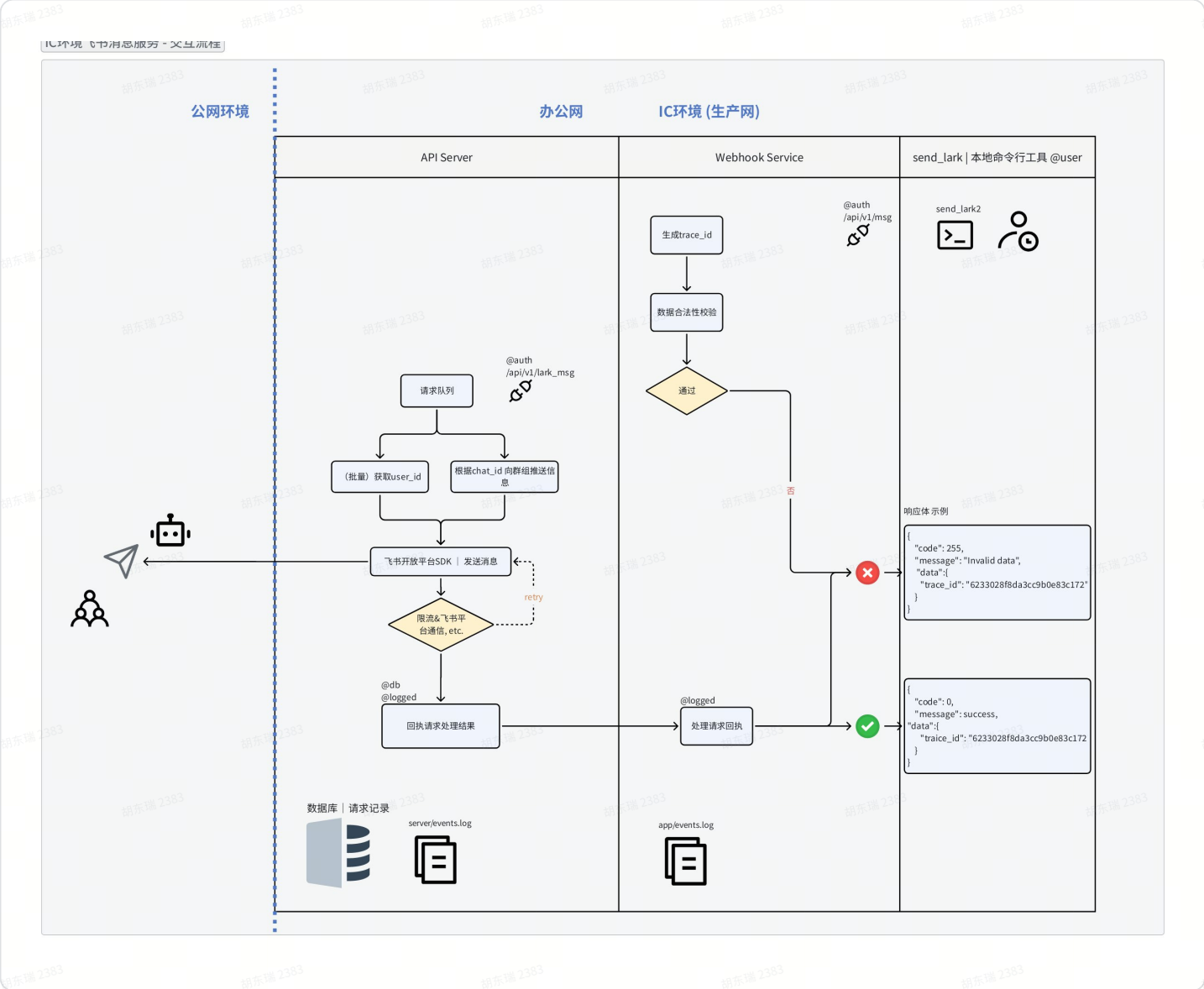
简介

send_lark2 飞书消息服务是一个针对 **IC研发环境** 向 **办公网环境** 发送飞书消息的微服务工具。

- 服务通过Webhook 收敛研发网络环境内的消息请求并调用飞书API接口实现发送。
- 出于对研发网络数据安全的考量，消息字符串长度受限可配置（默认2048字符），且消息记录可追溯。

> 🔥 **注意：** 服务假设研发环境内的Linux用户名 是 用户飞书企业账号的工作邮箱前缀；如与这有差异，请自行维护适配研发网用户账号 与 办公企业账号的 对应方式。

数据流程图



环境依赖

- 操作系统版本: **Centos 7.9, Rocky 8.10**
- 开发环境Python 版本：**python3.8.8+ (推荐: 3.12)**
- 支持 Docker容器镜像部署 和 Systemd 服务部署
- 推荐使用 uv 管理 Python开发环境依赖

工具安装及配置

1. 准备工作

1.1 网络安全策略

请联系企业IT自行维护： 加白放行 **办公网环境服务端** 与 **研发环境Webhook** 的网络策略

> server-host-01:5000 <-> webhook-host:5000

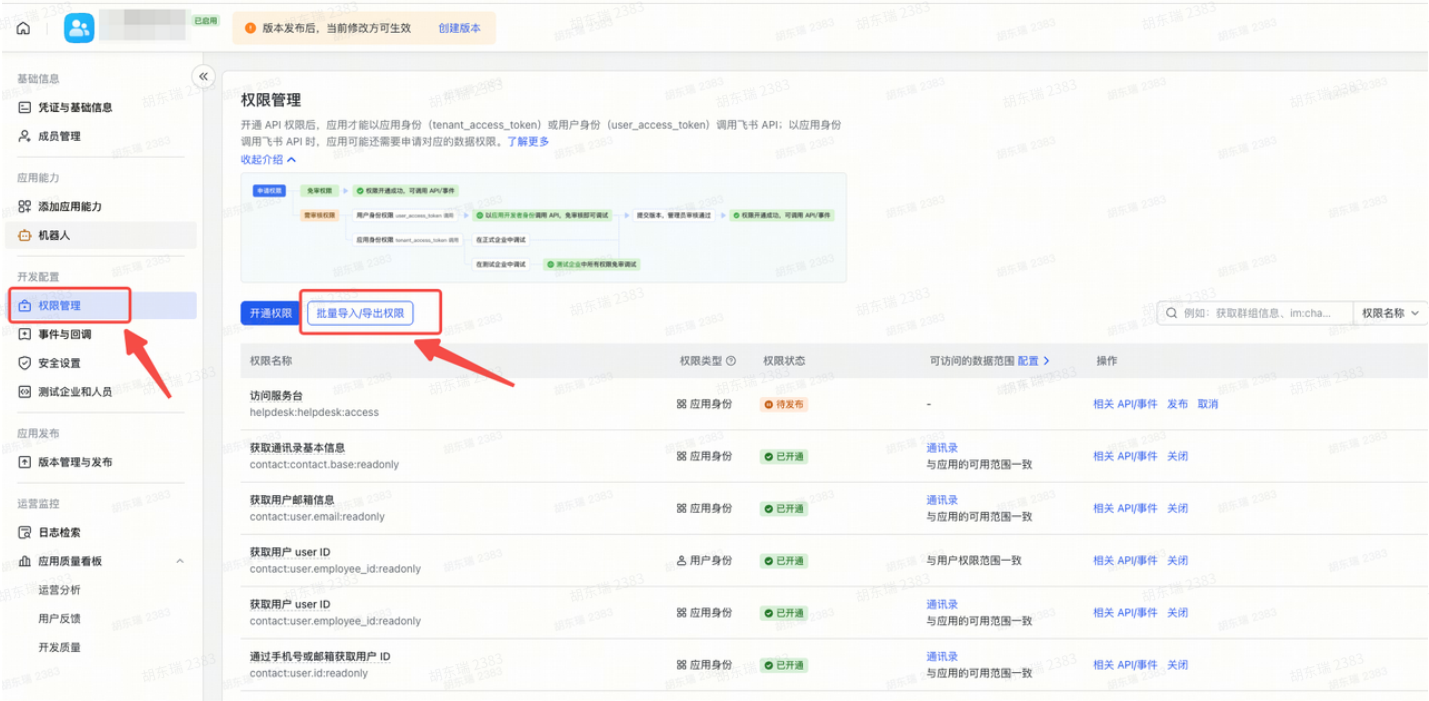
1.2 配置飞书机器人

1.2.1 创建飞书机器人应用

请参考 https://open.feishu.cn/document/develop-an-echo-bot/introduction 指引在你的企业飞书域中创建企业应用

1.2.2 导入API权限配置

前往**开发者后台** -> **开发配置** -> **权限管理**，将 `misc/api_access.json` 导入至**权限管理**页面



1.2.3 创建版本并发布，等待飞书企业管理员审核

> ⚠ **注意：**请确保应用为内部应用
>
> - 可用范围为 **所有员工**
> - **不要**勾选**对外共享**，避免用户推送消息至企业外飞书用户

2. 部署服务端

2.1 编译容器镜像

- 将 `./server` 视作服务端 工作路径，使用 `server/Makefile` 执行便捷安装操作。

代码块

```
cd server

# 更新Makefile 中的配置，如PORT, LARK_APP_ID, LARK_APP_SECRET, etc.

# 安装 Python依赖环境
make install

# 编译为容器镜像
make build

docker load < lark_msg_server:$VERSION.tar

docker images | grep lark_msg_server
```

2.2 准备工作路径配置文件

2.2.1 `docker-compose.yml`

> 默认服务端口 `$PORT` 为 `5000` ，请按需更新端口映射；
>
> 容器内默认sqlite数据库路径 `$DB_PATH` 为 `/app/db` ，推荐指向本地映射路径；
>
> 容器内默认日志路径 `$LOG_PATH` 为 `/tmp/lark_msg_service/log` ，可按需更新本地映射挂载；
>
> 容器内默认配置路径 `$CONF_PATH` 为 `/app/conf` ，推荐指向本地映射路径，在变更后可便捷重启服务。

docker-compose.yml

```yml
services:
  lark_msg_service:
    platform: linux/amd64
    image: lark_msg_service:$VERSION  # TODO
    restart: always
    ports:
      - "5000:5000" # TODO
    environment:
      - TZ=Asia/Shanghai
      - LOG_PATH=/tmp/lark_msg_service/log
    volumes:  # TODO
```

```
      - $LOG_PATH:/tmp/lark_msg_service/log  # TODO
      - $CONF_PATH:/app/conf
      - $DB_PATH:/app/db
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:$PORT/hi"]  # TODO
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 10s
```

2.2.2 更新配置文件

server/conf/config.ini

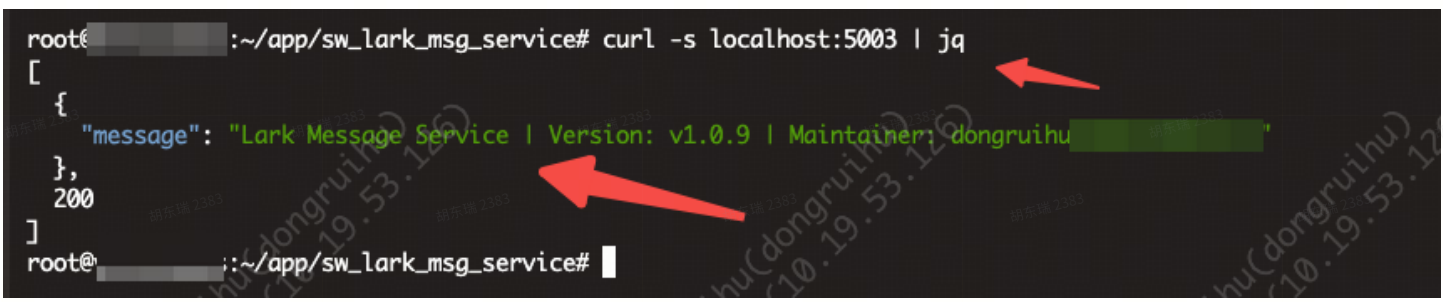
```
[default]
APP_ID=  # 飞书机器人应用鉴权参数
APP_SECRET=
TEMPLATE_ID=  # 飞书卡片消息模版ID
LOG_PATH=/tmp/lark_msg_service/log
USER_DOMAIN=email_domain.com  # 用户企业邮箱域名
SERVER_DOMAIN=open.feishu.cn
SERVER_PORT=5000
DATA_KEEP_ALIVE=60
```

2.3 运行/停止服务

> Make 指令集选项:
>
> - `run` | 按照docker-compose.yml 配置运行容器镜像
> - `stop` | 停止容器镜像服务
> - `debug` | 进入正在运行的容器镜像

容器服务拉起后，可以通过 `docker ps` 检查运行状态。

- 通过 `curl -s $HOST:$PORT | jq` 检查服务运行状态



3. 部署Webhook 服务

3.1 编译容器镜像

- 将 `./client` 视作Webhook 服务的工作路径，使用 `client/Makefile` 执行便捷安装操作。

代码块

```
# 安装 Python依赖环境
make install

# 编译为容器镜像
make build

docker load < send_lark_webhook_$VERSION.tar

docker images | grep send_lark
```

3.2 准备工作路径配置文件

3.2.1 `docker-compose.yml`

> 默认服务端口为5000，请按需更新端口映射；
>
> 容器内默认日志路径为 `/tmp/send_lark_webhook/log`，可按需更新本地映射挂载；
>
> 容器内默认配置路径为 `/app/conf`，推荐指向本地映射路径，在变更后可便捷重启服务。

docker-compose.yml

```yaml
version: 'v1.0'
services:
  send_lark_webhook:
    platform: linux/amd64
    image: send_lark_webhook:v1.1.3
    restart: on-failure
    ports:
      - "5000:5000" # TODO
    environment:
      - TZ=Asia/Shanghai
      - LOG_PATH=/tmp/send_lark_webhook/log
    volumes:
      - /tmp/send_lark_webhook/log:/tmp/send_lark_webhook/log # TODO
      - /Users/bytedance/PycharmProject/send_lark2/client/conf:/app/conf # TODO

```

3.2.2 更新配置文件

client/config.ini

```ini
[default]
APP_ID=app-id-test # 飞书机器人鉴权参数 更新为对应的APP_ID/SECRET
APP_SECRET=app-secret-test
LOG_PATH=/tmp/send_lark_webhook/log # 指定日志路径
SEMAPHORE_COUNT=100    # FastAPI协程数，用于控制请求并发
DEBUG_MODE=true

SERVER_HOST=127.0.0.1 # 更新为办公网服务端信息
SERVER_PORT=5000

# For cli - send_lark2脚本侧配置，可选
CLIENT_PORT=__CLIENT_PORT__
PAYLOAD_SIZE=2048
QUERY_TIMEOUT=20

CLIENT_HOST=localhost
```

3.3 运行/停止服务

> Make 指令集选项:
>
> - `run` | 按照docker-compose.yml 配置运行容器镜像
> - `stop` | 停止容器镜像服务
> - `debug` | 进入正在运行的容器镜像

容器服务拉起后，可以通过 `docker ps` 检查运行状态。

- 通过 `curl -s $HOST:$PORT | jq` 检查Webhook服务运行状态
- 通过 `curl -s $HOST:$PORT/hi | jq` 进行与服务端的连通性检查

4. （可选）命令行工具 send_lark2

可以将 `scripts/` 下的脚本工具完成配置后安装至公共路径，便利用户在命令行中调用。

- 安装详情见 `scripts/README.md`
- 可以参考**附录一、导入&创建飞书卡片** 配置默认 & 自定义卡片模版
- 用户使用可参考 `doc/send_lark2飞书消息服务_用户手册.pdf`

日志管理

服务端/Webhook 的默认日志路径为 `/tmp/$APP/log`

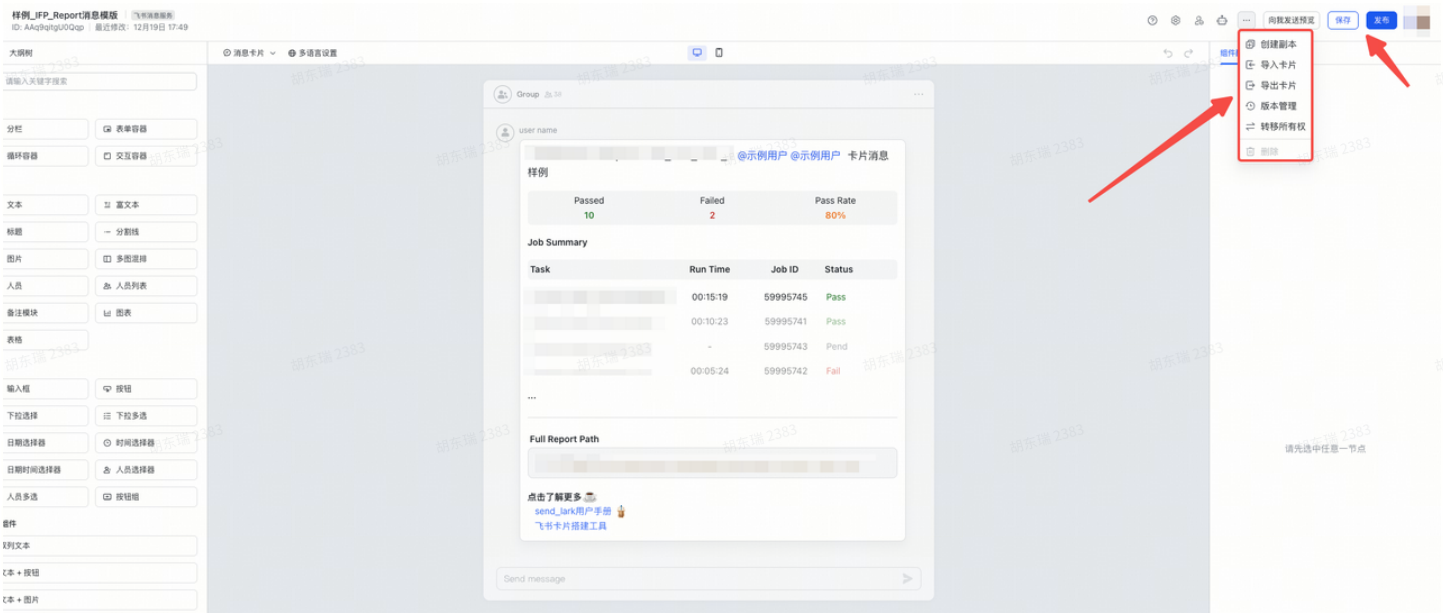
`access.log` - API访问日志

`app.log.*` - 事件日志，用于追溯消息请求发送情况。可以使用 `trace_id` 便捷检索排查；日志留存默认备份，前往 `conf/logging_config.json` 进行配置

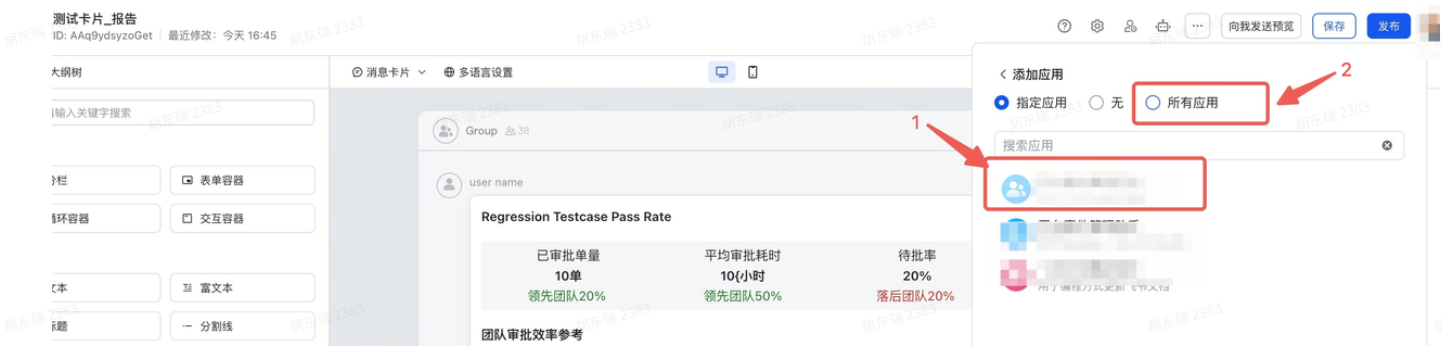
附录

附录一、导入&创建飞书卡片

1. 前往 飞书卡片消息搭建工具 (https://open.feishu.cn/cardkit)，参照下图导入 `misc/msg_template.card` 为默认模版。



2. 自定义消息卡片请参照飞书开放平台（https://open.feishu.cn/document/feishu-cards/quick-start/send-message-cards-with-custom-bot）自行创建并调试卡片格式
3. 搜索并添加已发布的自定义机器人/应用。如果无法搜到之前创建的机器人应用，可选择 **“添加应用”** -> **“所有应用”**



4. **保存** 并 **发布**后，找到卡片变量，提供/分享 卡片ID

5. 命令行工具 `send_lark2` 中，使用 `-c` 声明 JSON 格式变量内容，可以是文本文件或字符串。使用示例:

代码块

```
send_lark2 -i AAqd3vZhVkYrm -c variables.json -r ou_xxxxxxx
```

附录二、API接口文档

除了命令行工具，脚本软件也可以调用API接口发送消息。

1. 请求头参数

请求方法: POST

参数名	说明	是否必填	示例值
Content-Type	指定请求数据格式	**是**	application/json
x-app-secret	应用密钥，用于身份验证和授权	**是**	uuid
x-app-id	应用ID，标识调用API的应用	**是**	app-xxxxxx
x-username	发起请求的用户名	**是**	具体用户名，可通过$(whoami)获取

2. 请求体格式

- **普通文本消息** `/lark/api/v1/send_lark_msg`

请求体采用JSON格式，包含以下字段：

参数名	说明	是否必填	示例值
title	消息标题	**是**	`"This is a title."`
content	消息内容	**是**	`"这是一条飞书消息\\n这是第二行 {user: user1}"` （通过 `{user: username}` @用户）
recipients	接收者列表	**是**	["user1", "user2"]
header_color	消息头部颜色	**否**	green

- **模版卡片消息** `/lark/api/v1/send_card`

请求体采用JSON格式，包含以下字段：

参数名	说明	是否必填	示例值
card_id	卡片模版ID	**是**	`"AAq9qitgU0Qqp"`
content	文本格式的JSON变量 (str)	**是**	使用 `json.dumps()` 将字典转换为字符串即可
recipients	接收者列表	**是**	["user1", "user2"]

3. 响应体格式

响应体同样采用JSON格式，包含以下字段：

参数名	说明	示例值
return_code	返回码，0表示成功	0
message	消息描述，表明调用结果	success
trace_id	跟踪ID，与请求中的跟踪ID一致	16位uuid

4. 错误码

错误码	错误描述	可能原因	解决方法
4xx	用户调用	应用密钥或应用ID错误	检查x-app-secret和x-app-id是否正确填写
422	请求体格式错误	请求体没有通过格式校验	检查3.3中请求体格式是否正确
5xx	服务内部错误	请求体不符合规定的JSON格式	检查请求体的JSON格式是否正确，各字段是否齐全

5. 代码示例

代码块

```
# 研发环境webhook服务地址 & 飞书机器人鉴权参数
export SERVICE_HOST=127.0.0.1
export SERVICE_PORT=5000
export APP_ID=app-xxxxxx
export APP_SCRET=xxxxxxx

title="This is a title"  # 标题
content="test message content"  # 内容
recipient=$(whoami) # Linux 用户名

curl -s -X POST -H "Content-Type: application/json" \
   -H "x-app-secret: $APP_SECRET" \
   -H "x-app-id: $APP_ID" \
   -H "x-username: $(whoami)" \
   -d "{\"title\":\"$title\", \"content\": \"$content\", \"recipients\": [\"$recipient\"], \"trace_id\": \"13asd743f\", \"header_color\": \"green\"}" \
   http://$SERVICE_HOST:$SERVICE_PORT/lark/api/v1/send_lark_msg | jq

```

JSON

```
{
  "return_code": 0,
  "message": "success",
  "trace_id": "13asd743f"
}
```

- `return_code`：返回码，`0` 表示API调用成功。
- `message`：消息描述，返回码为非0值时，返回错误信息

- `trace_id`：跟踪ID，与请求中传递的跟踪ID一致，方便在日志记录和调试过程中，追踪整个请求的处理流程，便于定位问题。